

ICLAB 2019 Final Exam

Paper Based Exam

Total Score : 104

Name:

Student ID:

1. [6%] Please draw the diagram of setup and hold time check if a register in CLK1 domain is writing signal into a register in CLK2 domain under the set_multicycle_path timing constrain:

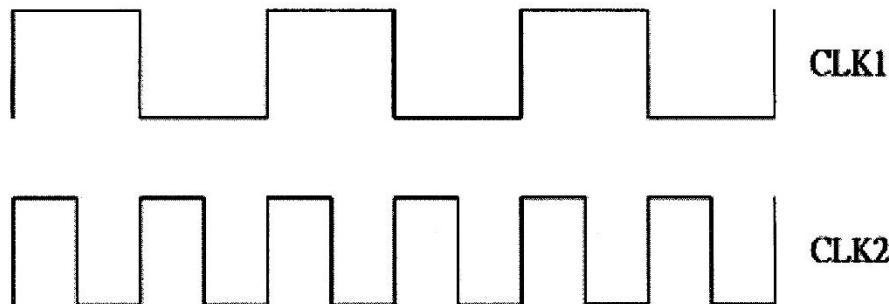
(1) (2 points)

set_multicycle_path 1 -end -from CLK1 -to CLK2

set_multicycle_path 0 -hold -start -from CLK1 -to CLK2

Given that the waveform relationship of clk1 and clk2 is:

(if the first rising edge of CLK1 is perfectly aligned to the first rising edge of CLK2 and the period of CLK1 is twice longer than the period of CLK2)



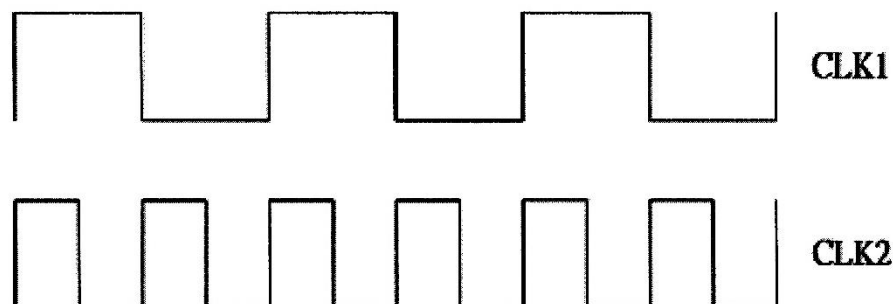
(2) (4 points)

set_multicycle_path 3 -start -from CLK1 -to CLK2

set_multicycle_path 2 -hold -start -from CLK1 -to CLK2

Given that the waveform relationship of clk1 and clk2 is:

(if the first rising edge of CLK1 is perfectly aligned to the first rising edge of CLK2 and the period of CLK1 is twice longer than the period of CLK2)



2. [9%] Please explain the main difference between double flop synchronizer and XOR double flop synchronizer if we use them in a CDC (clock domain crossing) design and why do we need to use synchronizer in a CDC design.
3. [10%] Actually, we can use report_power to get our power report in the design compiler. However, we usually get our power report by using PrimeTime. Why should we using PrimeTime to get the power report? What kind of file should we prepare if we want to get the power report by using PrimeTime?
4. [6%] Please list two problems which will happen when a signal propagates between two power domains at different voltage levels. What kind of circuits should we use to deal with these problems?
5. [20%] For (a) to (g), please fill in the blank
For (h) to (k), please answer the following question

(h) : how many packed array is declared in ALU.sv? how many bits is used to generate these packed array?

(i) : please indicate the problem of type FSM_STATE.

(j) : please indicate the problem of type STAGE.

(k) : please rewrite Piece.v so that we can put it inside the program of PATTERN.sv

Note: a-1 and a-2 are related to each other. Please write them both on your answer sheet.

Please clearly label your answer from (a-1), (a-2) to (k). (a)-(g), (h)(k) [2%] (i), (j) [1%]

```

1  `timescale 1ns/100ps
2
3  `include "Ustertype_PKG.sv"
4  `include "INF.sv"
5  `include "PATTERN.sv"
6  `include "ALU.sv"
7
8
9  module TESTBED;
10
11  parameter simulation_cycle = 20;
12  reg SystemClock;
13
14  INF    inf(a-1);
15  PATTERN test_p(.clk(SystemClock), .inf(b));
16  ALU    dut_b(.clk(SystemClock), .inf(c));
17
18  //----- Generate Clock -----
19  //    some clk generated block definitioin
20  //    ( you don't need to modify )
21  //-----
22
23  endmodule

```

Testbed.sv

```

21  `ifndef USTERTYPE
22  `define USTERTYPE
23
24  g usertype;
25
26  typedef enum { S_IDLE = 3'd0,
27                S_a    = 3'd1,
28                S_b    = 3'd2,
29                S_c    = 3'd3,
30                S_d    = 3'd4
31              } FSM_STATE;
32
33  typedef enum { S_e = 2, S_f = 3, S_g, S_h = 4 } STAGE;
34
35  typedef logic [3:0] type_A;
36  typedef logic [7:0] type_B;
37  typedef type_B[1:0] type_C;
38
39  typedef struct packed {
40      integer    temp_int;
41      type_C     c;
42      logic[15:0] d;
43  } type_struct ;
44
45  typedef union packed{
46      type_struct    mem_struct;
47      type_A         [15:0] mem_a;
48      type_B         [7:0]  mem_b;
49      type_C         [3:0]  mem_c;
50  }my_union_type;
51
52  /* some other type definition */
53
54  end g
55
56  `endif

```

Ustertype_PKG.sv

```

22  interface INF(a-2);
23      g usertype(f);
24      logic rst_n;
25      DATA D;
26      logic in_a_valid;

```

```

22 interface INF(a2);
23     e usertype f;
24     logic rst_n;
25     DATA D;
26     logic in_a_valid;
27     logic in_b_valid;
28     logic in_c_valid;
29
30
31     logic out_valid;
32
33     // .....
34
35
36     d PATTERN(
37         input clk,
38         output rst_n
39         // ..... //
40     );
41
42
43     d ALU(
44         input clk, rst_n
45         // ..... //
46     );
47
48
49 endinterface
50

```

INF.sv

```

2 module ALU(input clk, C inf);
3     e usertype f;
4
5     logic [7:0] temp1;
6     integer temp2[15:0];
7     bit [3:0][7:0] temp3[1:10];
8
9     my_union_type my_union;
10    type_A get_a;
11    type_B get_b;
12    type_C get_c;
13    type_struct get_struct;
14
15
16    //-----
17    // ( you don't need to modify here )
18    //-----
19
20
21 endmodule
22

```

ALU.sv

```

1 `ifdef RTL
2     `define CYCLE_TIME 15
3 `endif
4 `ifdef GATE
5     `define CYCLE_TIME 15
6 `endif
7
8
9
10 program automatic PATTERN(input clk, b inf);
11     e usertype f;
12
13     //-----
14     // ( you don't need to modify here )
15     //-----
16
17 endprogram
18
19

```

PATTERN.sv

```

1 always@(negedge clk)begin
2     if(inf.out_valid)begin
3         $display("out_valid is high");
4     end
5 end
6

```

Piece.v

6. [6%] What are code coverage and functional coverage?
7. [9%] If we want to check whether our design hits the specification, we can write the assertion in the check file. There are two signals, A and B, in the design. Please finish three assertions as the below.
 - (1) A happens then 2 or more cycles later B happens
 - (2) If A happens, then B must happen at the same cycle.
 - (3) If A happens, then B must happen at the next cycle.

Assertion1: assertion property (@(posedge clk)) ;
 Assertion2: assertion property (@(posedge clk)) ;
 Assertion3: assertion property (@(posedge clk)) ;
8. [8%] Why do we need to add pad filler and filler cell in APR flow? Please explain the reason respectively.
9. [8%] Please explain what is crosstalk, and list at least three solutions.
10. [4%] Briefly explain what are (a) power analysis [4%] and (b) rail analysis
11. [4%] (a) IR drop can cause the chip to fail due to (at least 3)?
 [4%] (b) What can we do to solve IR drop?

12. [7%] True or False:

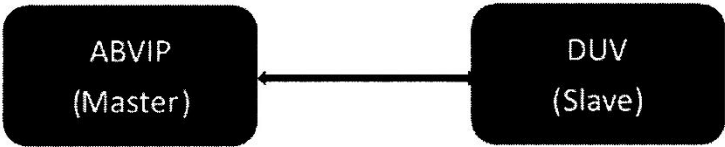
- (1) () Formal Verification could totally replace simulation-based verification in current IC Design flow.
- (2) () We normally write our formal verification syntax in our Pattern.sv to setup our verification environment.
- (3) () Formal Verification is a verification method that could be applied only in design written in system Verilog
- (4) () Generally, we use cover , assert , and assume to setup our verification environment.
- (5) () Assume properties tell formal tool what kind of stimulus is legal, so only the matched stimulus will be considered to verify our design.
- (6) () Glue Logic is an alternative method to make our formal properties (such as assume, assert or cover) simple and easy to read.
- (7) () In code coverage, we could comment out few of some assume properties to check whether our verification environment is over-constrained or not.

Hint: code coverage is a verification methodology to test whether each statement/branch/expression ha be covered.

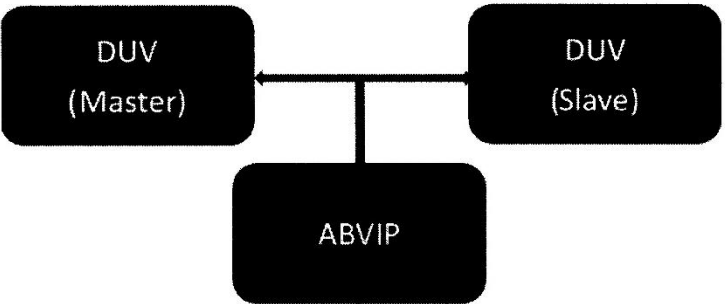
13. [3%] Fill in the Blanks:

In Verification IP usage, there are 3 kind of Verification-Oriented configuration to set up our AXI-verification environments. (Master Mode, Slave Mode and Monitor Mode)

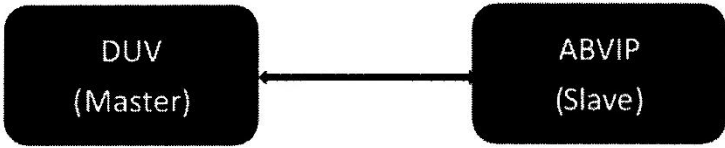
Hint: ABVIP stands for Abbreviation Based Verification IP
DUV stands for Design Under Test



1. ____ Mode



2. ____ Mode



3. ____ Mode